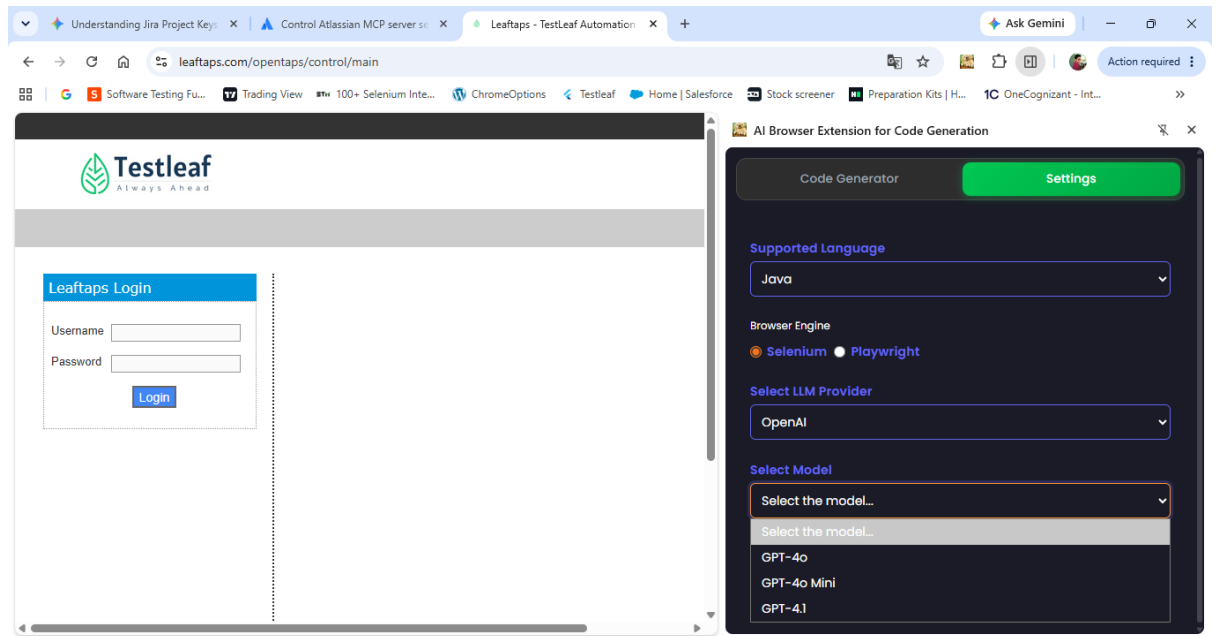


Week-3-Day1 – Assignment

1. Add “GPT-4.1” model to existing list of Open AI provider

Added GPT-4.1 to the model dropdown in `popup.js`

Added GPT-4.1 pricing in `pricing.js`



2. Implement Playwright with Typescript:

Ask: Act as a senior architect and understand the entire project code. This will be useful for further enhancement or changes in the project

Plan: Project is having implementation for Selenium with java. When i select Language as Java and Selenium as Browser Engine in the Setting tab. Selenium with Java code is getting generated successfully. I want to implement for TypeScript as Selected Language and Playwright as Browser engine without impacting the existing implementation for Selenium Java. Analyze the impacted files and functions. I need to review and approve. Then proceed with implementation in Agen mode

Agent: Proceed with the implementation

Understanding Jira Project Keys x Control Atlassian MCP server se x LeafTaps - TestLeaf Automation x + Ask Gemini

leafTaps.com/opentaps/control/main

Software Testing Fu... Trading View 100+ Selenium Inte... ChromeOptions Testleaf Home | Salesforce Stock screener Preparation Kits | H... 1C OneCognizant - Int...

Testleaf
Always Ahead

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

Code Generator Settings

Supported Language
TypeScript

Browser Engine
☒ Selenium ☐ Playwright

Select LLM Provider
InceptionLabs

Select Model
Mercury 2

InceptionLabs API Key
.....

Understanding Jira Project Keys x Control Atlassian MCP server se x LeafTaps - TestLeaf Automation x + Ask Gemini

leafTaps.com/opentaps/control/main

Software Testing Fu... Trading View 100+ Selenium Inte... ChromeOptions Testleaf Home | Salesforce Stock screener Preparation Kits | H... 1C OneCognizant - Int...

Testleaf
Always Ahead

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

Code Generator Settings

☒ Feature File ☒ Page Class

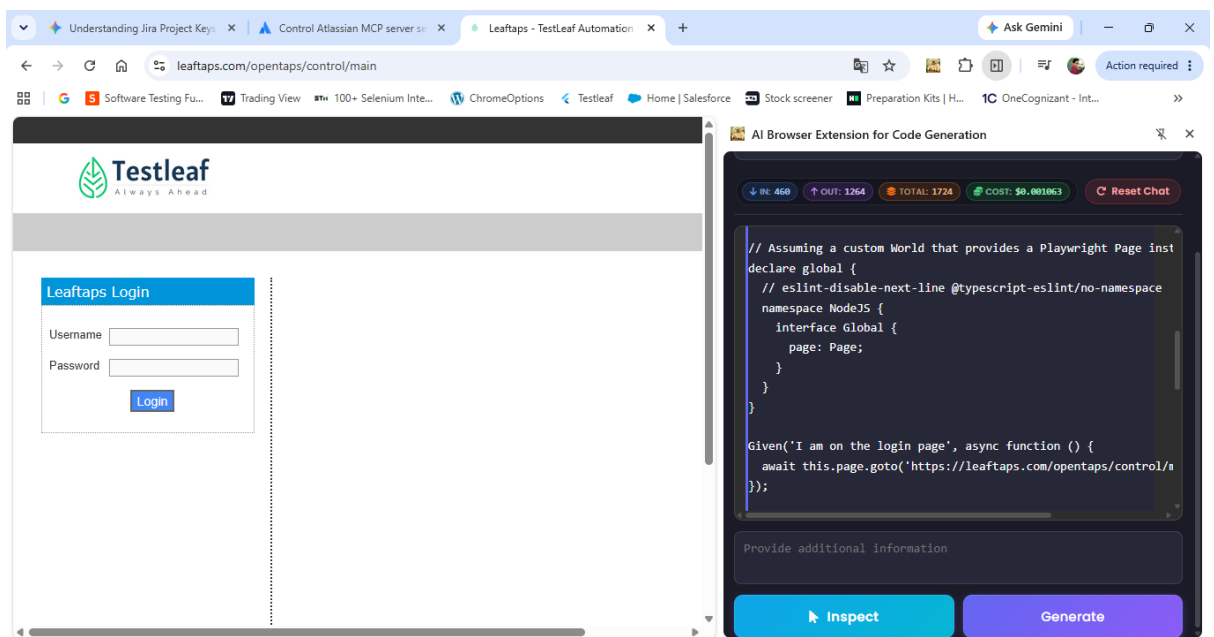
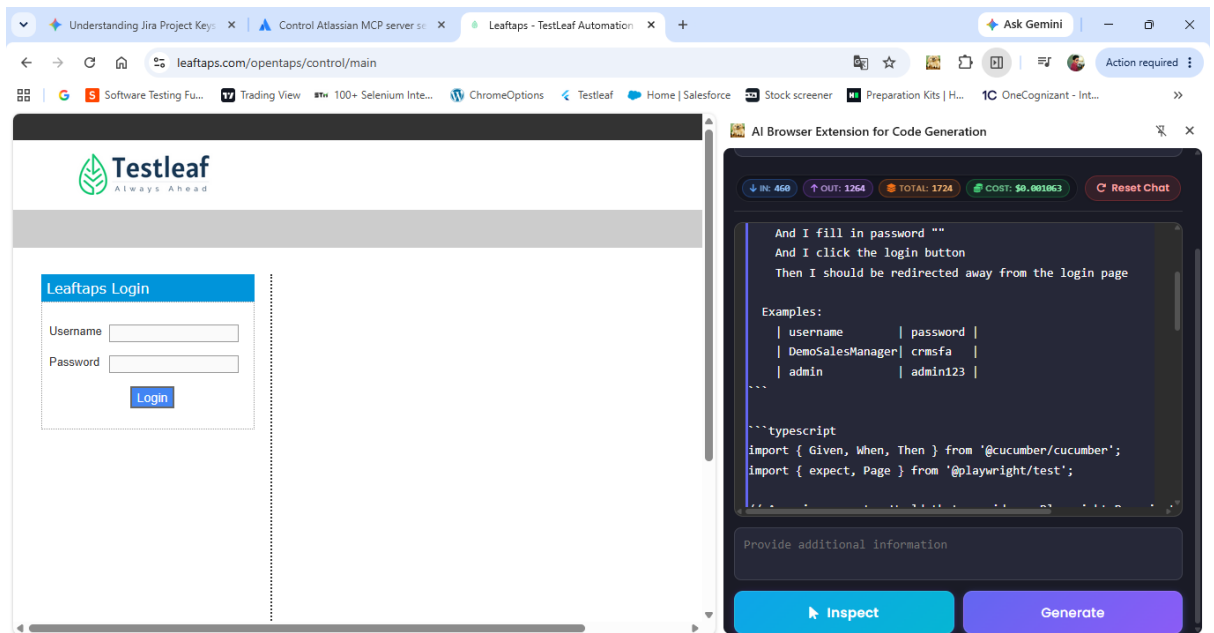
IN: 460 OUT: 1264 TOTAL: 1724 COST: \$0.001963 Reset Chat

Copy

```
---gherkin
Feature: LeafTaps login functionality
  As a user, I want to log into the LeafTaps application using

  Scenario Outline: Successful login with valid credentials
    Given I am on the login page
    When I fill in username ""
    And I fill in password ""
    And I click the login button
    Then I should be redirected away from the login page

  Examples:
```



``gherkin

Feature: LeafTaps login functionality

As a user, I want to log into the LeafTaps application using valid credentials.

Scenario Outline: Successful login with valid credentials

Given I am on the login page

When I fill in username ""

And I fill in password ""

And I click the login button

Then I should be redirected away from the login page

Examples:

```
| username      | password |  
| DemoSalesManager| crmsfa   |  
| admin         | admin123 |  
``
```

```
``typescript
```

```
import { Given, When, Then } from '@cucumber/cucumber';  
import { expect, Page } from '@playwright/test';
```

```
// Assuming a custom World that provides a Playwright Page instance as `this.page`  
declare global {
```

```
  // eslint-disable-next-line @typescript-eslint/no-namespace
```

```
  namespace NodeJS {
```

```
    interface Global {
```

```
      page: Page;
```

```
    }
```

```
  }
```

```
}
```

```
Given('I am on the login page', async function () {  
  await this.page.goto('https://leftaps.com/opentaps/control/main');  
});
```

```
When('I fill in username {string}', async function (username: string) {  
  const usernameInput = this.page.locator('input#username');  
  await expect(usernameInput).toBeVisible();  
  await usernameInput.fill(username);  
  await expect(usernameInput).toHaveValue(username);  
});
```

```
When('I fill in password {string}', async function (password: string) {  
  const passwordInput = this.page.locator('input#password');  
  await expect(passwordInput).toBeVisible();  
  await passwordInput.fill(password);  
  await expect(passwordInput).toHaveValue(password);  
});
```

```
When('I click the login button', async function () {  
  const loginButton = this.page.locator('input.decorativeSubmit');  
  await expect(loginButton).toBeVisible();
```

```
    await expect(loginButton).toBeEnabled();
    await loginButton.click();
  });
```

```
Then('I should be redirected away from the login page', async function () {
  // The login page URL is known; after a successful login it should change.
  await
  expect(this.page).not.toHaveURL('https://leaftaps.com/opentaps/control/main');
});
``
```